

Department of Computer Science
CS403NOD – Node.js

Lab Assignment – 4

Advanced Search, Filter & Sort API

Semester: BCA VII

Lab No.: 4

Objective

After completing this task, you should be able to:

- Combine multiple query parameters together using AND logic.
- Implement case-insensitive, partial text search.
- Sort results by any field, in ascending or descending order.
- Validate query input and respond with correct HTTP status codes for bad input.
- Combine route parameters and query parameters in the same request.
- Understand how real-world REST APIs commonly implement filtering, search, and sorting.

Tasks

Task 1 – Setup

Continue inside your existing folder. Create a new file.

advanced-server.js

Start from this minimal skeleton only — everything else in this task, you will write yourself. `const http = require('http');`

`const url = require('url');`

```
const students = [  
  // add at least 8 objects here – see Task 2  
];
```

```
const server = http.createServer((req, res) => {  
  res.setHeader('Content-Type', 'application/json');  
  const parsedUrl = url.parse(req.url, true);  
  const pathName = parsedUrl.pathname;  
  const query = parsedUrl.query;
```

```
  // your logic goes here
```

```
});
```

```
server.listen(3000, () => console.log('Server running on port 3000'));
```

Task 2 – Expand Your Data

Create at least 8 student objects, each with id, name, course, and a new numeric field:

marks. `{ id: 1, name: "Aman", course: "BCA", marks: 72 }`

Include a mix of at least 2 different courses and a wide spread of marks (some high, some low)

— your filtering and sorting logic will be tested against this spread.

Task 3 – Multi-Field Filtering (AND logic)

Support course and minMarks as independent, optional query parameters that combine with AND

logic.

`/students?course=BCA&minMarks=70`

Should return only students whose course matches AND whose marks are greater than or equal to minMarks. All four combinations must work correctly:

- only course given
- only minMarks given
- both given together
- neither given — return everyone

Hint: Start from the full array. If query.course exists, filter it down. If query.minMarks exists, filter the ALREADY-filtered result again — not the original array. Chaining .filter() calls is the cleanest way to do this.

Task 4 – Partial Text Search

Add a search query parameter that matches if a student's name CONTAINS the given text anywhere — not just an exact match — and must be case-insensitive.

`/students?search=an` → should match “Aman”, “Chandan”, “Sanya”, etc.

Hint: convert both the name and the search text to the same case before comparing, and check whether one contains the other.

Task 5 – Sorting (Ascending & Descending)

Add sort and order query parameters.

`/students?sort=marks&order=desc` → highest marks first

`/students?sort=name&order=asc` → alphabetical order

Rules:

- If order is missing, default to ascending.
- Only "name" and "marks" are valid values for sort.
- Any other value for sort (e.g. sort=xyz) must return status 400 with a clear JSON error message — do not silently ignore it.

Task 6 – Combine Everything

All of the above must work together correctly in a single request.

`/students?course=BCA&minMarks=60&search=a&sort=marks&order=desc` Think carefully about the order of operations: filter first (course, minMarks, search), then sort the filtered result — not the original array.

Task 7 – Input Validation

If minMarks is provided but is not a valid number (for example `/students?minMarks=abc`), your server must NOT crash and must NOT silently return the wrong data.

Respond with status 400 and a JSON body clearly explaining what went wrong, for example: `{ "error": "minMarks must be a number" }`

Task 8 – Bonus Challenge (optional, hardest)

Support a route parameter version of the course filter, which can ALSO accept the same query parameters on top of it.

`/students/course/BCA?minMarks=60&sort=marks&order=desc`

This means your route needs to read a value from the path AND the query string at the same time, then apply the same filtering and sorting logic you already built in Tasks 3–6.

Testing Checklist

Before submitting, confirm each of these works correctly in the browser or

Postman: 1. /students — returns all students

2. /students?course=BCA — only that course

3. /students?minMarks=60 — only students at or above that mark

4. /students?search=a — partial, case-insensitive name match

5. /students?sort=marks&order=desc — correctly sorted, highest first

6. /students?sort=xyz — returns status 400 with an error message

7. /students?minMarks=abc — returns status 400 with an error message

8. /students?course=BCA&minMarks=60&search=a&sort=marks&order=desc — all combined correctly

Take a screenshot showing at least 4 of the above tests.

Save it as

advanced-output.png

README.md

Update your existing README.md to also include:

- Every query parameter your server supports, and what each one does
- One example URL for each parameter
- A short note (2–3 sentences) on how you handled invalid input

GitHub Submission

Use your existing repository — no need to create a new one.

Step 1

Open Terminal inside NodeJS-Lab.

Step 2

Run the following commands.

git add .

git commit -m "Lab 04 - Advanced Search, Filter and Sort API"

git push

Google Classroom Submission

Submit

- GitHub Repository Link
- (Optional) ZIP File

If You Face Any Problem

If you are unable to complete any task, **do not leave it incomplete without explanation.** Create a section in your **README.md** named:

Problems Faced

Mention:

- Task Number
- Error / Issue
- What you tried to solve it
- Screenshot (if applicable)

Example

Problems Faced

Task No.: Task 6

Issue:

Sorting worked, but it was sorting the full list instead of my filtered results.

Attempted Solution:

Realized I was calling .sort() on the original students array instead of the variable holding my filtered results. Fixed by sorting the filtered array.

Students who clearly document genuine problems will receive guidance during the next lab session.